Replication of MigrateLib: A Tool for End-to-End Python Library Migration

Namu Go
ng2349@nyu.edu
New York University Abu Dhabi
Abu Dhabi, Abu Dhabi, United Arab Emirates

Nelson Mbigili
nfm8340@nyu.edu
New York University Abu Dhabi
Abu Dhabi, Abu Dhabi, United Arab Emirates

Abstract

This report presents a replication study of *MigrateLib*, an end-to-end tool that automates Python library migration using Large Language Models (LLMs) and static/dynamic analysis. The original work demonstrated that while LLMs alone could correctly migrate 57% of code, the full MigrateLib tool improved effectiveness to 86% of files through post-processing. Our replication evaluated these claims by: (1) manually auditing 10 randomly selected migrations from the RQ-1.1 benchmark to verify the authors' categorization accuracy, and (2) fully reproducing the RQ-2.1 experiment, which evaluated how many migrations the tool can perform correctly on 717 real-world migration scenarios. Our manual evaluation of 10 randomly selected migrations largely agreed with the original authors' labels, with only minor discrepancies attributable to the nondeterministic behavior of LLMs and the variability in API usage. For RQ-2.1, we executed migrations on 717 scenarios using GPT-4o-mini and analyzed 710, excluding cases where pre-migration tests failed. Median correctness increased modestly across pipeline stages, from 81% after LLM-only migration to 82% after merge-skipped transformation and 84% after async transformation.

ACM Reference Format:
Namu Go and Nelson Mbigili. 2026. Replication of MigrateLib: A Tool for End-to-End Python Library Migration. In *NYU Abu Dhabi Software Analytics Replication Series*. NYU Abu Dhabi, UAE, 3 pages.

1 Introduction

Library migration in software development refers to changing one or more libraries that a certain project uses to other libraries for various reasons such as software security, library maintenance condition, or compatibility with the project architecture. The process entails checking all points in the project repository at which the former library is used and replacing them with the corresponding Application Programming Interface (API) calls in the new library. Since the procedure requires familiarity with both old and new libraries as well as manual work by the developers, it could be time-consuming and error-prone.

Contributing to the efforts to automate the process of library migration, Islam et al. [1] studied how effectively Large Language Models (LLMs) can perform automated migration. Building on PyMigBench [2], their previous work of making a Python library migration benchmark, they evaluated the correctness of LLM migrations against developers' migrations. Moreover, they built a Command-Line Interface (CLI) tool called MigrateLib that further automates the LLM migration pipeline. Islam et al. reported that *GPT-4o* , the best performing LLM out of 3 models they have tested with, could correctly perform 57% of migrations, and that MigrateLib showed complete correctness in 32% of migration scenarios without any human intervention.

Our work is a replication of MigrateLib [1], specifically the results of the following research questions:

- **RQ-1.1** How similar are the LLM migrations to the benchmark migrations?
- **RQ-2.1** How many migrations can MigrateLib perform correctly?

For RQ-1.1, we randomly select 10 migrations from the results of the paper and manually check them. For each migration scenario, we audit the original code, the developers' migration, and the LLM migration and evaluate how the authors have categorized the LLM migration results. We identify code changes originally categorized as correct that we would have categorized otherwise due to library-specific corner cases that LLM has failed to address.

For RQ-2.1, we reproduce the migration using the exact same pipeline of MigrateLib that the authors have used. We used GPT-4o-mini instead of GPT-4o due to financial considerations, and since we based our migration on the list of 717 scenarios found in the artifact of the paper, we skipped the candidate library collection process. We observe a higher overall correctness despite having used a less powerful LLM than the original MigrateLib, and we accredit this improvement to the general advancement of LLM technology since the time of the original experiment.

The results of our replication show that while LLM still occasionally fails to account for all intricacies and corner cases of every migration scenario, it displays potential for automating and accelerating the migration process. Furthermore, upon observing the improvement of LLM correctness in the timespan since the original paper, we conclude that the future enhancement of LLM will increase the usefulness and self-sufficiency of LLM-powered automated migration tools.

All artifacts related to this replication study are available online.[1]

2 Replication Study Details

2.1 Summary of Original Study

The original paper follows 3 distinct phases of experiment. First, the authors systematically investigate the performance of 3 different LLMs (GPT-4o, GPT-mini, Llama) on Python library migrations. Using a dataset of 314 migrations from PyMigBench, an artifact from their previous research [2], they give the same prompt to each LLM specifying the source library and the target library then observe the changes LLM makes on the pre-migration code. In this phase, they pose the following research questions:

CS-UH 2210, Abu Dhabi, UAE
2026.

[1] https://github.com/Nelsonmbigili/Replication_MigrateLib

- **RQ-1.1** How similar are the LLM migrations to the benchmark migrations?
- **RQ-1.2** How many migrations pass unit tests?

In the second phase, the authors build MigrateLib, an end-to-end CLI tool that complements LLM migration with needed pre- and post-processing. Namely, MigrateLib identifies all files in a given project that use the specified source library, and after migration, it adds back any code that LLM has skipped. Additionally, to address the possibility of dependents of the migrated files also needing changes, MigrateLib finds new asynchronous functions after migration, spots all functions in the project repository that use them, then makes them asynchronous.

Lastly in the third phase, the authors evaluate the real-world usefulness of MigrateLib by simulating migrations on 717 public project repositories, then running tests found in those repositories to validate migration correctness. This evaluation process answers the following research questions:

- **RQ-2.1** How many migrations can MigrateLib perform correctly?
- **RQ-2.2** How much manual effort is required to complete the incomplete migrations?

The full pre-and post-migration codes of 314 migrations in Phase 1, the complete scripts of the entire MigrateLib architecture, and the list of 717 public projects used in Phase 3 are all provided in the original study artifact.

2.2 Study Design

Our replication focuses on a subset of the original research questions, expressly RQ-1.1 and RQ-2.1. For RQ-1.1, we randomly select 10 migration scenarios from the artifact and manually check the reported results. Comparing the pre-migration code, human developers' migration, and LLM migration, we analyze and challenge the validity of the labels (correct, partially correct, incorrect, etc.) given to the LLM-migrated code.

For RQ-2.1, we run MigrateLib on the same 717 public projects as in the original simulation, and measure the correctness of the resulting codes. We execute the original scripts for both the migration and the correctness evaluation with minimal configuration changes. All such changes are documented in our Github Issues history, and the most important ones are as follows: (1) we change the LLM configuration from GPT-4o to GPT-4o-mini, due to financial considerations; (2) since the public project data collection step is not necessary for our replication, we implement the process of downloading each project repository, originally found in the 'data_collection' folder, in the main runner file ('mig_runner.py').

During the execution of MigrateLib, 7 projects were filtered out due to failing crucial pre-migration tests, ultimately leaving us with the results of 710 migrations.

3 Results

3.1 RQ-1.1

The LLM produced migrations that were largely consistent with benchmark implementations, with most cases classified correctly despite minor implementation differences.

Our manual evaluation of 10 randomly selected migration cases shows that the LLM successfully captured the essential migration logic in the majority of cases. While a small number of migrations omitted implementation details present in the developer-authored versions e.g., explicit handling of response encoding (Figure 1). In some cases the LLM performed wrong changes altering the code logic. But most of these were correctly labeled as the benchmark labels were sufficiently robust to account for the nondeterministic behavior of LLMs and the inherent variability in API usage patterns across libraries, resulting in classifications that remained appropriate even when surface-level differences were present.

We included manual evaluation reports for all 10 scenarios in our github repository.



Figure 1: In migration `aiohttp__ httpx__sk-415@harukabot__0611d16a`

3.2 RQ-2.1

Our replication study evaluated the success of migrations in passing pre-existing unit tests across 710 real-world scenarios. Overall, we found that a substantial portion of migrations successfully passed the unit tests, indicating that the LLM-generated changes were generally correct and compatible with existing code behavior. Specifically, 81% of scenarios passed all relevant unit tests following the initial LLM-only migration, increasing to 84% after applying the asynchronous transformation step. This modest improvement highlights the incremental benefit of post-processing transformations.

While these differences were expected given that our study used a different LLM model than the original paper, we observed a surprising detail: the gap between LLM-only migration and LLM with post-processing was significantly smaller in our replication. This suggests that the LLM performed remarkably well on its own, approaching the effectiveness of the more expensive model used in the original study. These results underscore the potential for LLM-based automation in library migrations, even when using smaller, more accessible models. The figure below illustrates the distribution of migration correctness scores across all scenarios using a violin plot, highlighting both the median performance and variability at each pipeline stage.

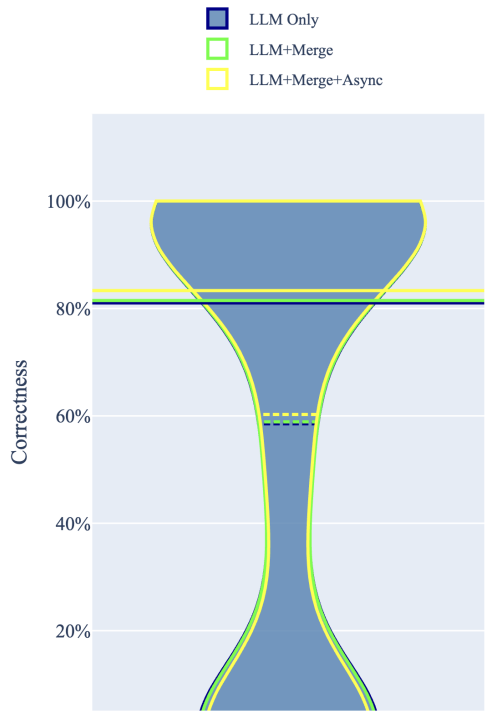


Figure 2: Distribution of migration correctness across 715 real-world scenarios in three steps.

4 Conclusion and Insights

This study replicated Research Questions 1.1 and 2.1 of the MigrateLib paper to verify the manual labeling accuracy of LLM migrations and the tool's end-to-end performance on real-world scenarios. Our replication confirmed that the original authors' labeling for RQ1.1 was robust, effectively accounting for the nondeterministic nature of LLMs and inherent variability in API usage and coding styles. For RQ2.1, even when using the more economical GPT-4o-mini model in 2026, we observed a significantly good performance in library migration tasks, with correctness reaching up to 84% after post-processing. These insights suggest that LLMs are increasingly viable for automated library migration, as rapid model evolution continues.

Reflecting on the replication process, the study was greatly facilitated by the authors' well-engineered codebase, which featured clear functional abstractions and high levels of automation. While the provided artifacts were excellent for reproduction, the replication could have been improved by adding brief descriptive comments on some code blocks, especially those that require familiarity with other parts of the codebase.

5 Statement of Contributions

Both authors of this study worked together in person on source code analysis, configuration adjustment, and migration selection for review for RQ-1.1. The task of executing MigrateLib was also divided, with Namu Go acquiring 320 migration results and Nelson Mbigili obtaining the rest.

Nelson Mbigili primarily worked on analyzing the code label statuses for RQ1.1 and wrote the results and conclusion section of this report.

Namu Go primarily worked on troubleshooting the Python dependency environment and modifying the 'mig_runner.py' file to fit the replication scope. He wrote Introduction and Replication Study Details in this report.

References

[1] Mohayeminul Islam, Ajay Kumar Jha, May Mahmoud, and Sarah Nadi. 2025. MigrateLib: a tool for end-to-end Python library migration. https://api.semanticscholar.org/CorpusID:282055672

[2] Mohayeminul Islam, Ajay Kumar Jha, Sarah Nadi, and Ildar Akhmetov. 2023. PyMigBench: A Benchmark for Python Library Migration. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. 511–515. doi:10.1109/MSR59073.2023.00075